对，按这 6 块来就清楚了。下面是可以直接带学生一步步做的版本。

---

1. VSCode 安装选项与打开后配置

安装版本

让学生下载：

```
Visual Studio Code User Installer
```

Windows 官方文档也推荐普通用户用 User setup，因为不需要管理员权限，更新也更顺。(Visual Studio Code)

Windows 安装时勾选项

安装到这一步：`Select Additional Tasks`

建议勾选：

```
Create a desktop icon
Add "Open with Code" action to Windows Explorer file context menu
Add "Open with Code" action to Windows Explorer directory context menu
Register Code as an editor for supported file types
Add to PATH
```

重点解释：

```
Create a desktop icon:
桌面生成 VSCode 图标，方便学生打开。

Open with Code:
以后可以右键文件夹，用 VSCode 打开项目。

Register Code as an editor:
让 .py、.md 这类文件可以默认用 VSCode 打开。

Add to PATH:
以后可以在终端里用 code 命令打开 VSCode。
```

如果学生不懂，不需要解释太多，直接说：

```
这一页能勾的基本都勾上，尤其是 Add to PATH 和 Open with Code。
```

Mac 安装

Mac 一般没有这些安装选项。

步骤：

```
1. 下载 VSCode
2. 打开下载好的 zip / dmg
3. 把 Visual Studio Code 拖进 Applications / 应用程序
4. 从 Applications 里打开 VSCode
```

重点提醒：

```
不要一直在下载文件夹里运行 VSCode，要拖到 Applications 里。
```

VSCode 打开后的配置

第一次打开后，只做这些，不要装一堆东西：

```
1. 选择主题：随便，学生喜欢就行
2. 安装 Python 扩展
3. 确认能打开 Terminal
```

安装 Python 扩展：

```
左侧 Extensions
搜索 Python
安装 Microsoft 发布的 Python 插件
```

解释：

```
Python 插件不是 Python 本体。
它只是让 VSCode 更好地识别、运行、提示 Python 代码。
```

打开终端：

```
顶部菜单 Terminal
选择 New Terminal
```

让学生知道：

```
VSCode 下面这个黑色/白色窗口就是终端。
以后我们会在这里运行 Python 文件。
```

---

2. Python 版本、安装选项与配置

用什么版本

建议用：

```
Python 3.12.x
```

原因：

```
1. 足够新
2. 兼容性比最新版本更稳
3. 对初学项目完全够用
```

Python 官网现在首页显示的最新版本是 Python 3.14.5，说明最新稳定版已经到 3.14 系列了。(Python.org) 但教学项目没必要追最新，尤其之后可能会装 OpenCV、MediaPipe 等库，用 3.12 更稳。

如果学生已经装了：

```
Python 3.11
Python 3.12
Python 3.13
```

都可以先用。 不建议用：

```
Python 2.x
Python 3.8 或更老
```

Windows 安装选项

安装 Python 时，最重要的是第一页：

```
一定勾选：
Add python.exe to PATH
```

然后点：

```
Install Now
```

不要选太复杂的 Customize install，除非你要改路径。

安装完成后，重新打开 VSCode 的 Terminal，输入：

```
python --version
```

如果显示类似：

```
Python 3.12.x
```

就成功。

如果 `python --version` 不行，试：

```
py --version
```

Mac 安装

Mac 推荐用 Python 官网安装包，不要一开始用 conda / brew，初学者会混乱。

安装后在 Terminal 输入：

```
python3 --version
```

如果显示类似：

```
Python 3.12.x
```

就成功。

Mac 上运行 Python 通常用：

```
python3 文件名.py
```

Windows 通常用：

```
python 文件名.py
```

或者：

```
py 文件名.py
```

VSCode 里选择 Python 解释器

安装完 Python 后，在 VSCode 里：

```
按 Ctrl + Shift + P / Mac 按 Cmd + Shift + P
输入 Python: Select Interpreter
选择 Python 3.12.x
```

如果列表里有多个 Python，选：

```
Python 3.12.x
```

不要选奇怪的 conda、base、Microsoft Store 之类，除非你明确知道它能用。

---

3. Hello World 环节

目标：

```
确认 VSCode + Python 都能正常工作。
```

创建课程文件夹

让学生在桌面或文档里建一个文件夹：

```
paper_keyboard_course
```

然后在 VSCode 里：

```
File
Open Folder
选择 paper_keyboard_course
```

如果弹出信任提示：

```
Do you trust the authors of the files in this folder?
```

选择：

```
Yes, I trust the authors
```

新建 hello.py

在 VSCode 左侧新建文件：

```
hello.py
```

写入：

```
print("Hello, Paper Keyboard!")
```

运行

Windows 终端输入：

```
python hello.py
```

如果不行：

```
py hello.py
```

Mac 终端输入：

```
python3 hello.py
```

看到：

```
Hello, Paper Keyboard!
```

就算成功。

你可以告诉学生：

```
到这里为止，我们已经完成了最小开发环境：
能写 Python 文件，也能运行 Python 文件。
```

---

4. 项目目录结构带解释

Hello World 跑通后，如果有时间，就开始建目录。

最终目录：

```
paper_keyboard_course/
  hello.py
  app/
  core/
  data/
    layouts/
    sessions/
  logs/
  scratch/
```

解释：

```
hello.py：
今天测试环境用的文件，后面可以保留，也可以不管。

app/：
放可以直接运行的程序。
比如之后的 replay_session_demo.py。

core/：
放核心逻辑。
```

```
比如之后的 input_engine.py。

data/:
放数据文件。

data/layouts/:
放键盘布局文件。
比如 keyboard_number_v2.json。

data/sessions/:
放输入过程记录文件。
比如 example_session.json。

logs/:
放每节课的项目日志。
比如 lesson_01_project_log.md。

scratch/:
放课堂练习、临时代码。
这里的代码不一定是正式项目代码。
```

用 VSCode 创建

建议初学者先用 VSCode 左侧文件树右键创建：

```
New Folder
New File
```

不用一开始强迫终端。

如果要用终端创建

Windows：

```
mkdir app
mkdir core
mkdir data
mkdir logs
mkdir scratch
mkdir data\layouts
mkdir data\sessions
```

Mac：

```
mkdir app
mkdir core
mkdir data
```

```
mkdir logs
mkdir scratch
mkdir data/layouts
mkdir data/sessions
```

你可以简单解释：

```
mkdir 就是创建文件夹。
```

不要讲太多命令行。

---

5. 项目日志结构

在 `logs/` 里新建：

```
lesson_01_project_log.md
```

内容可以直接让学生复制这个：

```
# 第 1 节项目日志：环境配置与项目整体理解

## 1. 今天完成了什么？

今天我完成了：

- 安装或确认 VSCode
- 安装或确认 Python
- 成功运行 hello.py
- 了解 Paper Keyboard 项目的基本目标
- 了解 session、layout、InputEngine 的作用
- 创建项目目录结构

## 2. Hello World 运行结果

我运行的文件是：

我的运行命令是：

程序输出结果是：

## 3. 这个项目最终想实现什么？

我的回答：

## 4. 摄像头、麦克风、InputEngine 分别负责什么？
```

```
摄像头：

麦克风：

InputEngine：

## 5. session JSON 和 layout JSON 分别记录什么？

session JSON：

layout JSON：

## 6. 当前版本为什么先假设 finger_id = 1？

我的回答：

## 7. 今天我还不太明白的问题

问题 1：

问题 2：

问题 3：
```

课堂上可以带他填这些：

```
第 1 部分：今天完成了什么
第 2 部分：Hello World 运行结果
第 3 部分：项目最终想实现什么
```

其他部分课后补。

---

6. 最后的可选小练习

如果还有时间，在 `scratch/` 里新建：

```
read_frame_demo.py
```

写：

```
frame = {
    "frame_id": 2,
    "t": 0.033,
    "fingers": [
```

```
            {
                "finger_id": 1,
                "x": 106.0,
                "y": 82.8
            }
        ],
        "tap": {
            "audio": True
        }
    }

    finger = frame["fingers"][0]

    x = finger["x"]
    y = finger["y"]
    audio = frame["tap"]["audio"]

    print("x =", x)
    print("y =", y)
    print("audio =", audio)
```

运行：

Windows：

```
python scratch/read_frame_demo.py
```

或者：

```
py scratch/read_frame_demo.py
```

Mac：

```
python3 scratch/read_frame_demo.py
```

预期输出：

```
x = 106.0
y = 82.8
audio = True
```

然后做三个小改动：

```
1. 把 x 改成 50，再运行
2. 把 y 改成 30，再运行
3. 把 audio 改成 False，再运行
```

问学生：

```
x 和 y 表示什么？
audio=True 表示什么？
audio=False 表示什么？
frame["fingers"][0] 取到的是什么？
```

答案：

```
x 和 y 表示手指在纸面上的坐标。
audio=True 表示这一帧检测到了敲击。
audio=False 表示这一帧没有检测到敲击。
frame["fingers"][0] 取到的是这一帧里的第一个手指；当前版本里就是假设的食指。
```

这一节课最低目标就定为：

```
VSCode 装好
Python 装好
Hello World 跑通
项目目录建好
项目日志建好
```

小练习是加分项，不是必须完成。